DAY 1 · FULL PROGRAMME

Advanced Machine Learning in Production

From leakage-free pipelines to deployed models: one loan-default case study, end to end.

Day 1 | 4 Hours | 240 Minutes | 4 Modules

Trainer: Muhamad Majed Tuah Bin Mohd Hafizuddin · 23 October 2026

Programme Goals & Objectives

MISSION

Core Purpose

Most organisations already use AI, yet fewer than half of AI projects reach production.

This programme closes the gap between a model that works in a notebook and a service that stays accurate, explainable and monitored after go-live.

Every exercise uses open-source tools you can adopt at no licence cost.

Key Principle: a model only creates value once it can be trusted in production.

OUTCOMES

Key Learning Outcomes

By the end of these 4 hours, you will be able to:

- ✓ (a) Choose classic ML, deep learning or an LLM
- ✓ (b) Detect leakage; build a leakage-free pipeline
- ✓ (c) Organise training into reproducible scripts
- ✓ (d) Tune with Optuna and track runs with MLflow
- ✓ (e) Explain predictions with SHAP in plain language
- ✓ (f) Detect drift with PSI and KS; set alert thresholds
- ✓ (g) Serve a model with FastAPI inside Docker
- ✓ (h) Monitor in production; map the flow to Azure ML

Your 4-Hour Roadmap

Four modules, one case study, a checkpoint at the end of every block

0:00–0:15

Welcome & Ice Breaker

15 MIN

- Introductions
- Objectives & ground rules
- "Would you ship it?"
- Environment check

CHECKPOINT
First notebook cell runs

0:15–1:10

Trustworthy Data & Leakage-Free Pipelines

55 MIN

- ML vs DL vs LLM
- Data quality & leakage
- Hands-on: find the leaks
- Pipelines & time splits

CHECKPOINT
Leak-free pipeline on a time split

1:10–2:05

Production Training, Tuning & Tracking

55 MIN

- Notebook to scripts
- Grid / random / Bayesian
- Hands-on: Optuna
- MLflow tracking

CHECKPOINT
Best trial visible in MLflow

2:05–3:00

Explainability & Drift Monitoring

55 MIN

- SHAP global & local
- Business narrative
- PSI & KS drift tests
- Anomalies & thresholds

CHECKPOINT
Explanation + PSI table

3:00–4:00

Serving, Monitoring & Cloud

60 MIN

- FastAPI + Docker
- Six-panel dashboard
- Azure ML demo
- Action plan & wrap-up

CHECKPOINT
curl returns a prediction

Why This Programme Exists

Adoption is high, but production is the bottleneck

78%

of organisations used AI in 2024, up from 55% a year earlier

Stanford AI Index, 2025

< 50%

of AI projects reach production; prototype to production takes ~8 months

Gartner via Wilkinson, 2024

294

published papers across 17 fields were affected by data leakage

Kapoor & Narayanan, 2023

63%

of employers say skill gaps are the biggest barrier to transformation

World Economic Forum, 2025

59%

of Malaysian organisations have deployed or are piloting AI

Aon, 2026

5%

of Malaysian organisations are confident they can hire enough AI talent

Aon, 2026

Sculley et al. (2015): real-world ML systems carry large hidden technical debt. Today we practise the habits that keep that debt small.

Ice Breaker: Would You Ship It?

Poll: vote A, B or C before the reveal

SCENARIO

99% accuracy

on a loan book where only 1% of borrowers default. Your manager wants it live on Monday.

- **A** Ship it, 99% is excellent
- **B** I need to see more metrics first
- **C** Reject it outright

REVEAL

A model that always says "no default" scores 99% and catches zero defaulters.

Per 1,000 loans (always predicts "no default")

990 True negative	0 False positive
10 Missed defaults	0 Caught defaults

- Accuracy 99%
- Recall 0%
- Business value none

Seen in our own Session 02 notebook: 99.2% accuracy, 0 of 2 test defaults caught. Choose metrics that match the business cost: recall, precision, PR-AUC.

One Case Study, the Full Production Lifecycle

A loan-default model travels through every stage today

1. **Choose the approach** — ML vs DL vs LLM
2. **Leakage-free pipeline** — scikit-learn
3. **Modular training** — Python + YAML
4. **Tune & track** — Optuna + MLflow
5. **Explain decisions** — SHAP
6. **Watch for drift** — PSI · KS · Isolation Forest
7. **Serve as an API** — FastAPI + Docker
8. **Monitor & scale** — Dashboard + Azure ML

ENVIRONMENT CHECK

- ✓ Python 3.11, Git and Docker Desktop installed
- ✓ Repository cloned from the students branch

TERMINAL

```
$ cd advance-ml-fama && jupyter lab
# open day1/01_ai_ml_dl_llm_landscape/
#      ai_ml_landscape_student.ipynb  -> run cell 1
```

MODULE 1

Trustworthy Data & Leakage-Free Pipelines

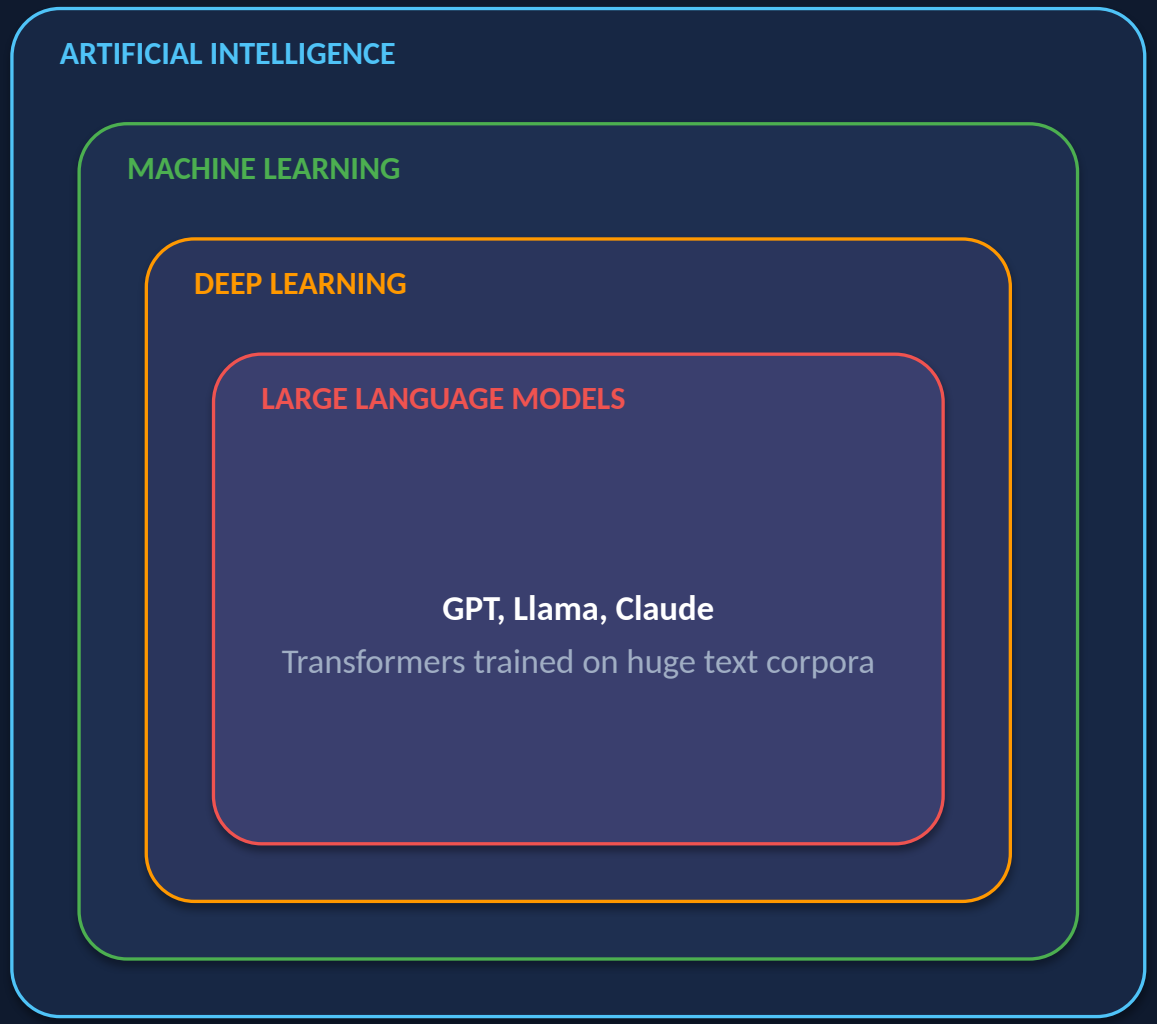
55 minutes | 0:15 – 1:10 | Outcomes (a) and (b)

IN THIS MODULE

1	Choosing the right approach	10m
2	Data quality and data leakage	10m
3	Hands-on: finding leakage	15m
4	Leakage-free pipelines & validation	20m

AI, ML, DL and LLMs: Nested, Not Competing

Each layer is a specialised subset of the one around it



AI — Any system that mimics intelligent behaviour: rules, search, planning or learning.

ML — Learns patterns from examples. Gradient boosting and random forests dominate tabular data.

DL — Multi-layer neural networks that learn features from raw images, audio and sequences.

LLM — Deep learning models for language: summarise, answer, draft and reason over text.

Choosing the Right Approach

Let the shape of the data, not the hype, pick the tool

Business problem	Data shape	Best fit	Why
Predict loan default	Tabular rows	Classic ML (gradient boosting)	Accurate, fast, explainable with SHAP
Segment customers with no labels	Tabular, no target	Unsupervised ML (clustering)	Finds groups without labelled examples
Forecast 7-day electricity demand	Time series	Classic ML / forecasting	Lags and seasonality; cheap to retrain
Detect defects on a conveyor camera	Images	Deep learning (CNN)	Learns visual features automatically
Transcribe customer service calls	Audio	Deep learning (speech model)	Pretrained models handle raw audio
Answer staff questions from policy PDFs	Unstructured text	LLM + retrieval (RAG)	Grounds answers in your own documents

Rule of thumb: most enterprise data is tabular, so start with classic ML. Reach for deep learning or LLMs when the data is images, audio or free text.

Quick poll: cover the 'Best fit' column and let the room call each answer.

Five Dimensions of Data Quality

Check them before you trust any metric

Integrity

Schema and types are as expected

LOAN DATA
credit_score arrives as text

Completeness

Missing values are limited and understood

LOAN DATA
8.8% of annual_income missing

Accuracy

Values are correct and realistic

LOAN DATA
11.9% of incomes outside IQR fences

Timeliness

Data is fresh enough for the decision

LOAN DATA
Bureau score pulled 6 months ago

Validity

Values respect business rules

LOAN DATA
Negative employment_years

WHY IS IT MISSING? The answer changes the fix

MCAR Missing completely at random

e.g. A form field dropped by a system glitch

Fix: Simple imputation is fine

MAR Depends on other observed columns

e.g. Income blank more often for self-employed

Fix: Impute using the related columns

MNAR Depends on the missing value itself

e.g. High-debt applicants skip the income field

Fix: Add a was_missing flag; investigate

Four Ways Information Leaks Into Training

Leakage gives optimistic validation scores and poor production performance

Target leakage

A feature is created after, or because of, the outcome.

LOAN EXAMPLE

days_to_default, post_approval_status_flag

FIX

Keep only features known at decision time

Train-test contamination

Preprocessing is fitted on all rows before the split.

LOAN EXAMPLE

Scaler or imputer fitted on the full dataset

FIX

Split first; fit inside a Pipeline

Temporal leakage

A random split lets the model learn from the future.

LOAN EXAMPLE

2025 loans train a model tested on 2023

FIX

Time-ordered split or TimeSeriesSplit

Group leakage

The same entity appears in both train and test.

LOAN EXAMPLE

A repeat customer in both sets

FIX

GroupKFold or split by customer_id

scikit-learn guidance: split the data before any preprocessing, and keep every fitted step inside a Pipeline.

Hands-On: Find Every Leakage Source

15 minutes · day1/02_data_quality_leakage/notebook.ipynb

THE 12 CANDIDATE COLUMNS

customer_id	loan_amount
annual_income	employment_years
credit_score	days_since_last_payment
existing_loans	loan_type
region	loan_approval_date
days_to_default	post_approval_status_flag

Target: defaulted_in_next_30_days · 1,200 loans · 0.7% default rate

YOUR TASK

For every column ask: would I know this value at the moment the loan is approved?

SAFE	Known at decision time
RISKY	Usable only with care (split or as-of date)
LEAK	Only exists after the outcome

Then run the leaky and clean models in the notebook and compare their reports.

Answer Key: Every Leakage Source

Reveal after pairs have compared their answers

LEAK	`days_to_default`	Only exists once a default happens
	`post_approval_status_flag`	Recorded after approval, encodes the outcome
RISKY	`loan_approval_date`	Not a feature; use it to order the split by time
	`customer_id`	Not a feature; use it to group repeat customers
	`days_since_last_payment`	Safe only if measured as-of the application date
	`annual_income · credit_score`	Have gaps; impute inside the Pipeline, not before
SAFE	`loan_amount · employment_years`	Known when the application is submitted
	`existing_loans · loan_type · region`	Known when the application is submitted

The Leakage-Free Pattern

Same data, same model: only the order of operations changes

WRONG

Full dataset → Fit imputer & scaler on all rows → Random split → Train model → Evaluate

Test-set statistics and future loans shape training, so the score is optimistic and production disappoints.

RIGHT

Full dataset → Time-ordered split (oldest 80%) → Fit Pipeline on train only → Tune on validation folds → Evaluate once on newest 20%

The holdout looks like next month's applicants, so the score estimates real production performance.

Kapoor & Narayanan (2023); scikit-learn developers, Common pitfalls and recommended practices.

Building the Pipeline in scikit-learn

Every learned statistic lives inside one object fitted on training rows only

PYTHON

```
df = df.sort_values("loan_approval_date")
cut = int(len(df) * 0.8)              # oldest 80% = train
train, test = df.iloc[:cut], df.iloc[cut:]

num = ["loan_amount", "annual_income", "employment_years",
       "credit_score", "days_since_last_payment", "existing_loans"]
cat = ["loan_type", "region"]

num_pipe = Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())])
cat_pipe = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))])
pre = ColumnTransformer([("num", num_pipe, num), ("cat", cat_pipe, cat)])

model = Pipeline([("pre", pre),
                  ("clf", LogisticRegression(max_iter=1000))])
model.fit(train[num + cat], train["defaulted_in_next_30_days"])
```

fit() learns from train only

Medians, scales and categories never see the test rows.

One object, every stage

The same transforms run in validation, batch scoring and the API.

One artefact to ship

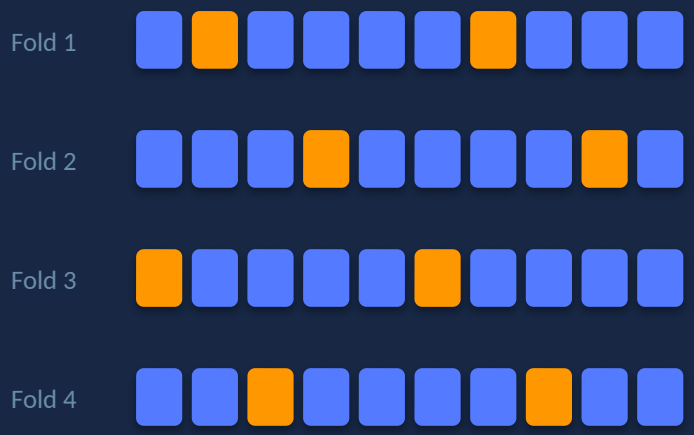
joblib.dump(model) saves preprocessing and model together.

Pick a Validation Strategy That Fits the Data

Train on blue, score on orange

Stratified K-Fold

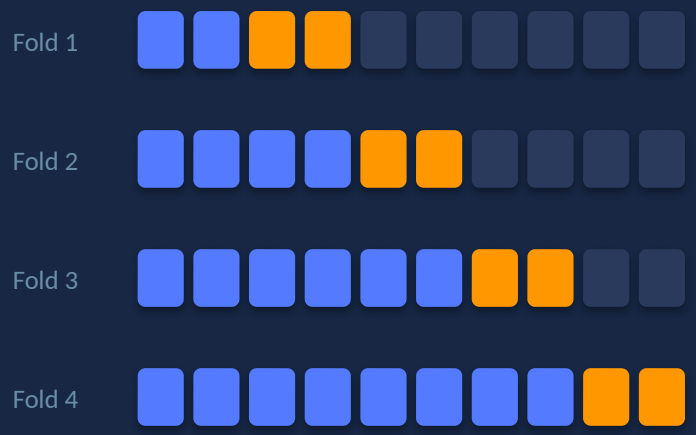
Independent rows; keeps the rare default rate in every fold.



Use for: cross-sectional snapshots, rare classes

TimeSeriesSplit

Ordered data: always train on the past, test on the future.



Use for: credit, fraud, demand, anything that predicts the future

GroupKFold

Repeat entities: all rows of one customer stay in one fold.



Use for: customers with many loans, patients, devices

Checkpoint & Quick Quiz

Confirm everyone can move on before starting the next module

CHECKPOINT

Your pipeline is scored on a time-ordered split with no leaked feature.

- ☐ Leaky columns removed
- ☐ Split happens before any preprocessing
- ☐ All preprocessing inside the Pipeline
- ☐ Metric reported on the newest 20% of loans

Q1

Why is fitting a scaler on the full dataset a problem?

Answer: Test-set statistics leak into training, so the validation score is optimistic.

Q2

When should you prefer a time-based split?

Answer: Whenever the model predicts future events from past data: credit, fraud, demand.

Q3

Which loan column is pure target leakage?

Answer: days_to_default: it only exists after the borrower has defaulted.

MODULE 2

Production Training, Tuning & Tracking

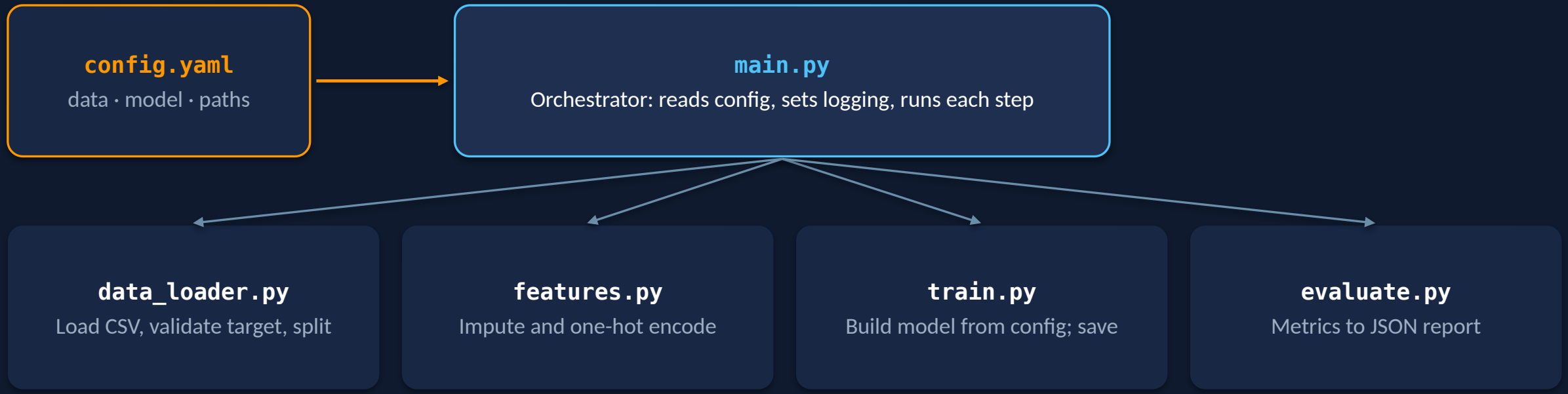
55 minutes | 1:10 – 2:05 | Outcomes (c) and (d)

IN THIS MODULE

1	From notebook to modular scripts	15m
2	Hyperparameter optimisation with Optuna	20m
3	Experiment tracking with MLflow	20m

From Notebook to Modular Scripts

day1/03_production_training/scripts: one responsibility per file



ARTIFACTS WRITTEN TO artifacts/

model.joblib
Fitted model

metrics.json
Accuracy, precision, recall, F1

training.log
Timestamped INFO log

- ✓ Each file is testable on its own
- ✓ Changes are reviewable in Git
- ✓ A scheduler can run it without Jupyter

Configuration, Seeds and Logging

Run the same command twice and get the same model

config.yaml

```yaml
data:
  path: data/production_training_sample.csv
  target_column: churn
  test_size: 0.2
  random_state: 42
model:
  type: logistic_regression
  random_state: 42
  max_iter: 500
paths:
  model_path: artifacts/model.joblib
  report_path: artifacts/metrics.json
  log_path: artifacts/training.log
```

TERMINAL

```
# run from the repository root
$ python day1/03_production_training/scripts/main.py \
    --config day1/03_production_training/scripts/config.yaml
→ ... | INFO | Starting training workflow
→ ... | INFO | Training complete. Metrics: {...}
```

Config, not code Change behaviour without editing Python

Fixed seeds random_state: 42 everywhere

Structured logs Timestamp | level | message

Saved artefacts Model + metrics + log per run

HANDS-ON

Run main.py once, then change `test_size` or `max_iter` in config.yaml and run again. Compare the two metrics.json files. Same config, same seed: identical result.

Parameters vs Hyperparameters

What the model learns versus what you decide

Model parameters

Learned from data during fit()

- ✓ Logistic regression coefficients
- ✓ Tree split thresholds
- ✓ Neural network weights

Hyperparameters

Set by you before training

- ✓ n_estimators, max_depth
- ✓ learning_rate, C (regularisation)
- ✓ min_samples_split

Baking analogy

The oven temperature and time are hyperparameters; you set them. How the ingredients combine inside is learned. Set them badly and the cake burns: the model overfits.

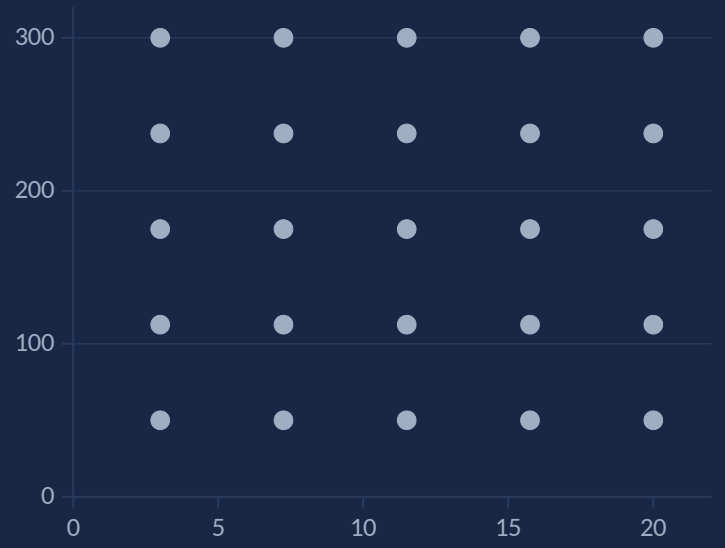
Golden rule

Tune on validation folds. Never let the test set choose hyperparameters.

Grid, Random and Bayesian Search

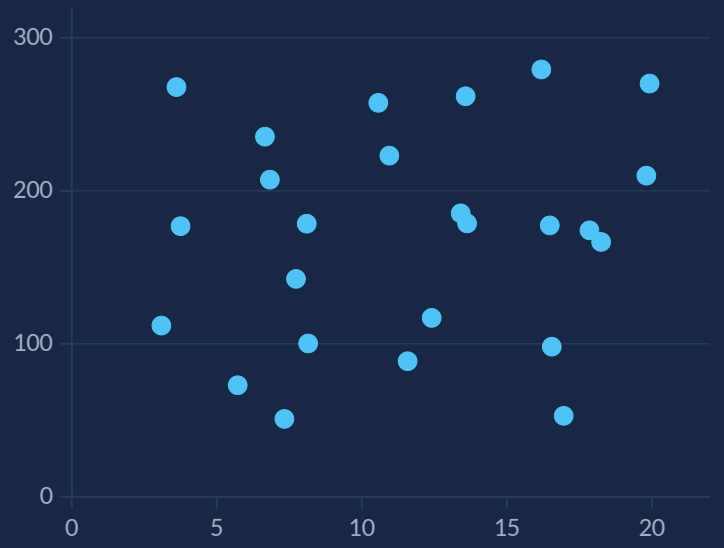
25 trials each over max_depth (3–20) and n_estimators (50–300)

Grid search



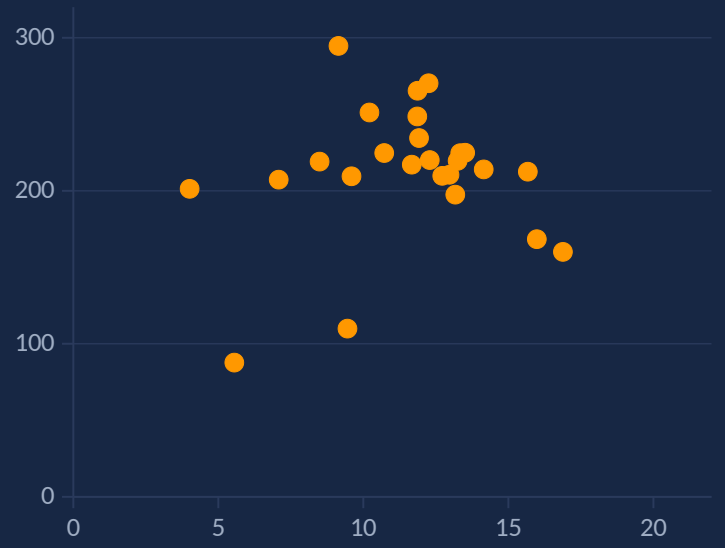
Tests every combination. Wastes trials on unimportant dimensions; cost explodes with more hyperparameters.

Random search



25 distinct values per axis. Bergstra & Bengio (2012): as good or better than grid for a fraction of the budget.

Bayesian (Optuna TPE)



Learns from past trials and concentrates new ones where scores are high.

Hands-On: Tuning With Optuna

10 minutes · day1/04_hyperparameter_optim/notebook.ipynb

PYTHON

```python
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 50, 300),
        "max_depth": trial.suggest_int("max_depth", 3, 20),
        "min_samples_split": trial.suggest_int("min_samples_split", 2, 15),
    }
    clf = RandomForestClassifier(**params, random_state=42)
    # score on validation folds, never on the test set
    return cross_val_score(clf, X_train, y_train,
                           cv=5, scoring="roc_auc").mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)
print(study.best_params, round(study.best_value, 3))
```

Trial
One evaluation of one set of hyperparameters

Study
The whole optimisation: all trials plus the best

Sampler
Chooses the next trial; TPE (Bayesian) by default

Pruner
Stops hopeless trials early to save compute

Stretch: plot_param_importances(study) shows which hyperparameter mattered most.

Experiment Tracking With MLflow

If it is not logged, it did not happen

PARAMS
max_depth=12, n_estimators=220

METRICS
test_auc, recall, f1

ARTIFACTS
model, plots, metrics.json

TAGS
git commit, data version, owner

PYTHON

```python
import mlflow, mlflow.sklearn

mlflow.set_experiment("Bootcamp_Optuna_Experiments")
with mlflow.start_run(run_name="Best_RF_Model"):
    mlflow.log_params(study.best_params)
    best = RandomForestClassifier(**study.best_params, random_state=42)
    best.fit(X_train, y_train)
    auc = roc_auc_score(y_test, best.predict_proba(X_test)[:, 1])
    mlflow.log_metric("test_auc", auc)
    mlflow.sklearn.log_model(best, "model")
```

Why track?

- ✓ Reproduce any past result
- ✓ Compare runs side by side
- ✓ Hand the exact model to deployment
- ✓ Audit trail for model risk teams

Comparing Runs in the MLflow UI

From many experiments to one candidate model

1. **Launch** `mlflow ui`
2. **Open** `127.0.0.1:5000`
3. **Select** `the experiment`
4. **Tick runs** `then Compare`
5. **Sort** `by test_auc`
6. **Register** `the best model`

Run	max_depth	n_estimators	test_auc	Decision
rf_baseline	None	100	0.781	Reference
optuna_trial_17	11	240	0.812	Candidate
optuna_trial_22	19	290	0.806	Slower, no gain

Illustrative values: your numbers will differ.

CHECKPOINT

main.py trains from the command line, and your best Optuna trial appears in the MLflow UI.

Checkpoint & Quick Quiz

Confirm everyone can move on before starting the next module

CHECKPOINT

main.py trains from the CLI and the best Optuna trial is in MLflow.

- ☐ Ran main.py with --config
- ☐ Changed config, reproduced a run
- ☐ Optuna study finished
- ☐ Best run logged with params, metric, model

Q1

Why does random search often beat grid search?

Answer: Few hyperparameters really matter; random search tries more distinct values of each.

Q2

Why must the test set never guide tuning?

Answer: It becomes part of training, so the final score is optimistic.

Q3

Name three things every MLflow run should log.

Answer: Parameters, metrics and artefacts (plus tags such as data version).

MODULE 3

Explainability & Drift Monitoring

55 minutes | 2:05 – 3:00 | Outcomes (e) and (f)

IN THIS MODULE

1	Explaining models with SHAP	20m
2	From explanation to business narrative	10m
3	Detecting data drift: PSI and KS	15m
4	Anomaly detection & alert thresholds	10m

Why Explainability Matters

"Your model denied my loan application. Why?"

LAW	**Regulatory compliance** GDPR Art. 22, MAS FEAT and similar rules expect explanations for automated decisions.
TRUST	**Stakeholder trust** Underwriters will not adopt a model they cannot question.
DEBUG	**Model debugging** Catch shortcuts, e.g. a vision model that learned to detect hospital rulers.
BIAS	**Fairness** Detect proxies for protected attributes such as age or gender.
AUDIT	**Auditability** Keep a reproducible evidence trail for past decisions.

THE TRANSPARENCY SPECTRUM

Linear / logistic
Glass box: read the weights

Decision trees / GAMs
Readable rules

Random forest / XGBoost
Needs post-hoc SHAP

Neural nets / LLMs
Black box: SHAP + LIME

Modern answer: train the best-performing model, then explain it post-hoc with SHAP. You no longer have to trade accuracy for compliance.

SHAP: Fair Credit for Every Feature

Shapley values from cooperative game theory (Lundberg & Lee, 2017)

The intuition

Features are players in a team; the prediction is the score. SHAP gives each feature its average marginal contribution across every possible team line-up.

$$f(x) = E[f(X)] + \Sigma \varphi_i$$

- ✓ Additivity: contributions sum exactly to the prediction
- ✓ Consistency: more influence never means a lower value
- ✓ Missingness: a feature with no effect gets exactly 0

PYTHON

```python
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

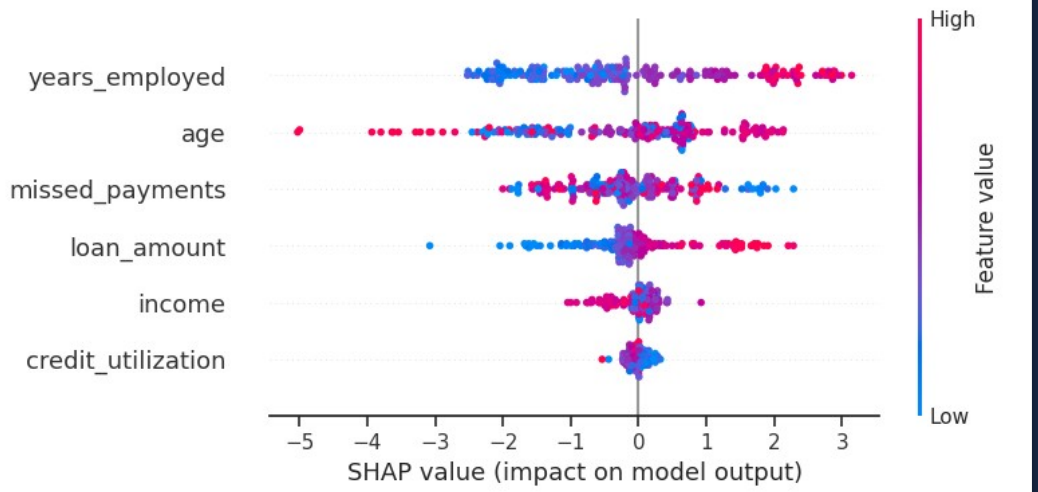
# Global: which features drive the portfolio?
shap.summary_plot(shap_values, X_test)

# Local: why this one applicant?
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0],
    base_values=explainer.expected_value,
    data=X_test.iloc[0]))
```

Global and Local Explanations

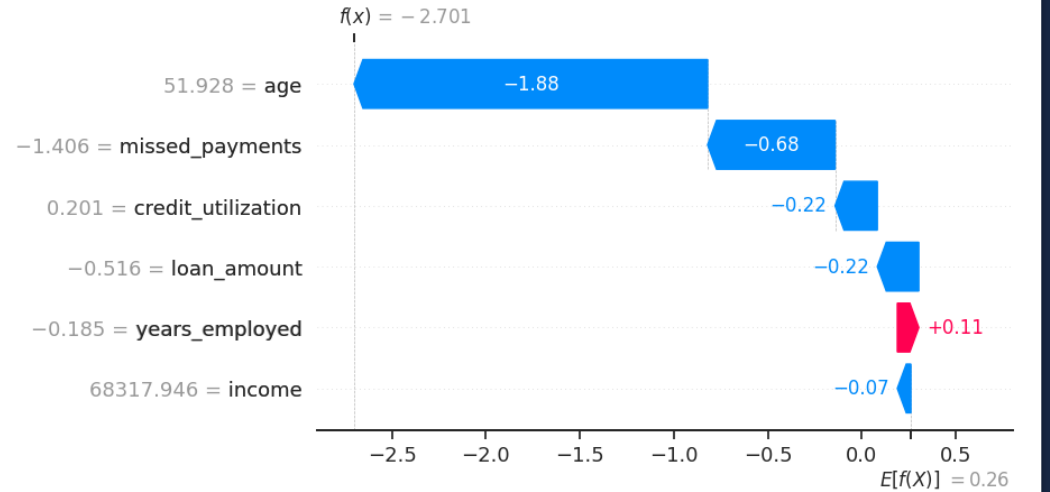
Same model, two questions: what drives the portfolio, and why this applicant?

GLOBAL · Beeswarm



years_employed, age and missed_payments drive portfolio risk. Red = high value; right = more risk.

LOCAL · Waterfall



Applicant #0: baseline 0.26 falls to −2.70 log-odds (≈6.3% default). Age and a clean record pull risk down: approve.

From Explanation to Business Narrative

Never hand raw SHAP numbers to a stakeholder

THE 3-STEP FORMULA

Observed value → Direction and size of impact → Actionable next step (counterfactual)

Worked example: adverse action

```
credit_utilization = 0.87   SHAP = +0.23
```

"High card utilisation (87%) was the main reason for the decline: it raised this applicant's estimated default risk well above the approval threshold."

Counterfactual: "Reducing utilisation below 30% would move this application to approval."

PAIR EXERCISE · 10 MIN

Write 3 sentences for one applicant from your waterfall plot:

- The main driver and its value
- Its direction and size, in plain words
- One action that would change the outcome

Swap with a neighbour: could a branch manager act on it with no ML background?

Three Kinds of Drift

The world moves on after training; your model does not

Feature drift

`P(X) changes`

The inputs shift.

A new digital channel brings much younger applicants.

DETECT WITH

PSI and KS on every input

Concept drift

`P(y | X) changes`

The rules change.

After rate rises, the same debt-to-income ratio now means higher risk.

DETECT WITH

Performance on newly labelled loans

Label drift

`P(y) changes`

The outcome rate shifts.

The default rate doubles from 3% to 6% in a downturn.

DETECT WITH

Predicted vs actual default rate

Measuring Drift: PSI and the KS Test

Training data vs one simulated production month

POPULATION STABILITY INDEX

$$PSI = \Sigma\ (A_i - E_i) \cdot \ln(A_i / E_i)$$

E_i, A_i = share of training and production rows in each of 10 training-quantile bins.

STABLE	WATCH	ALERT
< 0.10	0.10 – 0.20	≥ 0.20

KOLMOGOROV-SMIRNOV TEST

$$D = \max | F_{train}(x) - F_{prod}(x) |$$

p < 0.05 means the distributions differ. With thousands of rows KS flags tiny shifts, so pair it with PSI for size.

PSI BY FEATURE (SIMULATED MONTH)



Hands-On: Build Your PSI Table

day1/06_data_drift_anomaly/notebook.ipynb

PYTHON

```python
from scipy.stats import ks_2samp

rows = []
for col in numeric_features:
    psi = calculate_psi(train[col], prod[col])
    ks = ks_2samp(train[col], prod[col])
    rows.append({
        "feature": col,
        "psi": round(psi, 3),
        "ks_p": ks.pvalue,
        "status": "ALERT" if psi >= 0.2
                  else "WATCH" if psi >= 0.1
                  else "STABLE",
    })

drift = pd.DataFrame(rows).sort_values("psi", ascending=False)
```

Feature	PSI	KS p-value	Status
credit_utilization	0.301	< 0.001	ALERT
age	0.138	< 0.001	WATCH
annual_income	0.058	< 0.001	STABLE
credit_score	0.012	0.010	STABLE
employment_years	0.010	0.241	STABLE
loan_amount	0.004	0.637	STABLE

Notice: annual_income and credit_score fail the KS test ($p < 0.05$) but their PSI is small. Statistically different is not the same as big enough to act on.

Anomalies, Alert Thresholds and Cadence

Trainer demonstration: Isolation Forest, then agree a monitoring plan

TRAINER DEMO

Isolation Forest

Random splits isolate unusual rows in very few steps. No labels needed, so it works on live traffic from day one.

PYTHON

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05,
                      random_state=42)
iso.fit(X_train_numeric)

flags = iso.predict(X_new)   # -1 = anomaly
```

Cadence	Check	Threshold / action
Every request	Schema and range validation	Reject with HTTP 422
Daily	Volume, missing rate, anomaly rate	Anomaly rate > 2x baseline
Weekly	PSI and KS per feature	Watch 0.1 · Alert 0.2
Monthly	AUC and recall on labelled loans	Below SLO: retrain
Quarterly	Model review with risk team	Re-approve or retire

Alert only when a human must act. Everything else goes on the dashboard, not into someone's inbox.

Checkpoint & Quick Quiz

Confirm everyone can move on before starting the next module

CHECKPOINT

One plain-language explanation of a prediction, and a PSI table with drifted features flagged.

- ☐ SHAP summary and waterfall plots run
- ☐ 3-sentence explanation peer-reviewed
- ☐ PSI computed for every numeric feature
- ☐ ALERT and WATCH features flagged

Q1

What does SHAP additivity guarantee?

Answer: Feature contributions add up exactly to the prediction minus the baseline.

Q2

A feature has PSI = 0.27. What do you do?

Answer: Alert: investigate the cause and consider retraining.

Q3

Does Isolation Forest need labelled anomalies?

Answer: No. It is unsupervised, so it works on live data immediately.

MODULE 4

Serving, Monitoring & Cloud

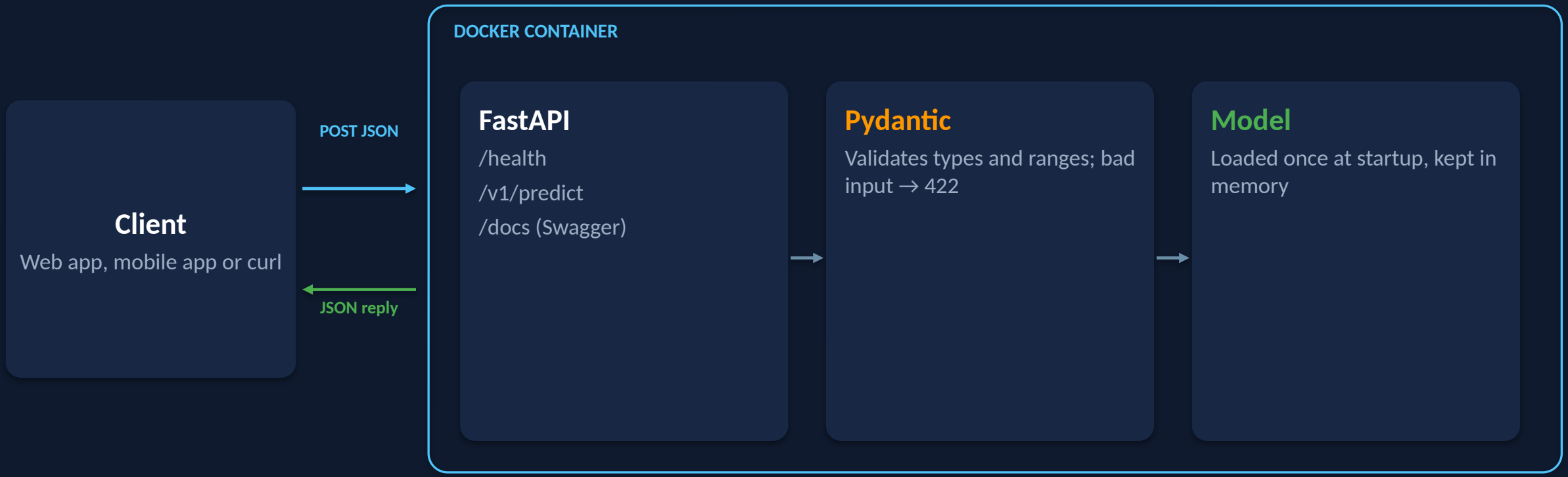
60 minutes | 3:00 – 4:00 | Outcomes (g) and (h)

IN THIS MODULE

1	Serving a model as a containerised API	20m
2	The MLOps dashboard	15m
3	Moving to Azure Machine Learning	15m
4	Action plan and wrap-up	10m

Serving a Model as an API

The API is the waiter: clients never walk into the kitchen



Validate everything
Never trust input from the internet.

Load once
Loading the model per request adds seconds of latency.

Health endpoint
Load balancers poll /health to route traffic.

FastAPI + Pydantic in 25 Lines

day1/07_api_serving_containers/app/main.py

PYTHON

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field

class PredictRequest(BaseModel):
    age: int = Field(..., ge=18, le=120)
    income: float = Field(..., gt=0)
    credit_score: int = Field(..., ge=300, le=850)

class PredictResponse(BaseModel):
    prediction: str
    probability: float

app = FastAPI(title="Bank Risk ML API")

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

@app.post("/v1/predict", response_model=PredictResponse)
async def predict(req: PredictRequest):
    if req.income > 60000 and req.credit_score > 650:   # stand-in model
        return PredictResponse(prediction="No Default", probability=0.1)
    return PredictResponse(prediction="Default", probability=0.85)
```

What to point out

- ✓ Field(ge=18, le=120) rejects age 7 with HTTP 422, before the model runs
- ✓ response_model fixes the output contract
- ✓ /v1/ in the path lets v2 run side by side
- ✓ Swagger UI appears at /docs for free

Production upgrade: replace the stand-in rule with joblib.load() of your Module 1 Pipeline at startup.

Hands-On: Build, Run and Call the Container

Image = frozen blueprint · Container = running copy

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app

# copy requirements first so the
# dependency layer is cached
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app
EXPOSE 8000
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

TERMINAL

```
$ cd day1/07_api_serving_containers/app
$ docker build -t loan-api:1.0 .
$ docker run -d -p 8000:8000 --name loan-api loan-api:1.0

$ curl -X POST http://localhost:8000/v1/predict \
    -H "Content-Type: application/json" \
    -d '{"age": 35, "income": 72000, "credit_score": 710}'
→ {"prediction":"No Default","probability":0.1}
```

CHECKPOINT

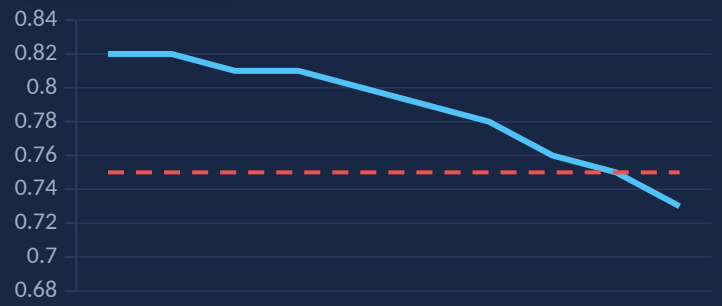
The container returns a prediction to your curl request. Windows PowerShell: use curl.exe and escape the inner quotes, or test from /docs.

The Six-Panel MLOps Dashboard

Four pillars on one screen: model, data, system and business

MODEL

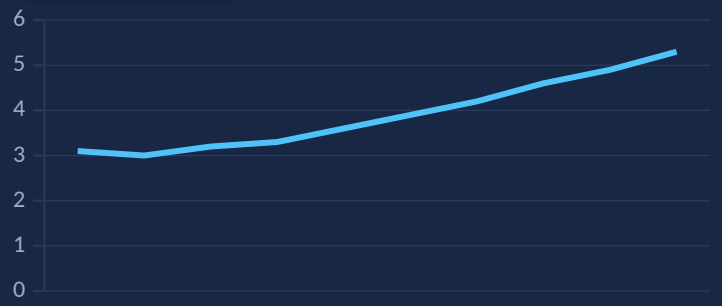
AUC over time vs SLO



Alert below 0.75

MODEL

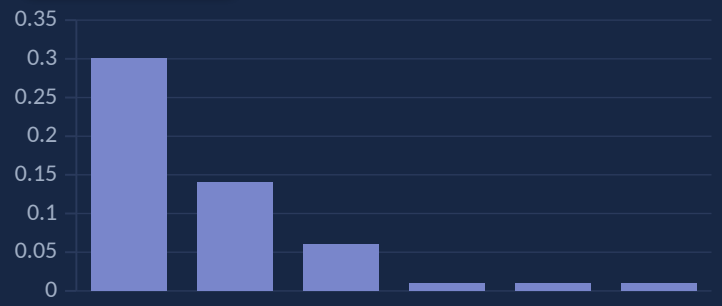
Predicted default rate



Compare with actuals

DATA

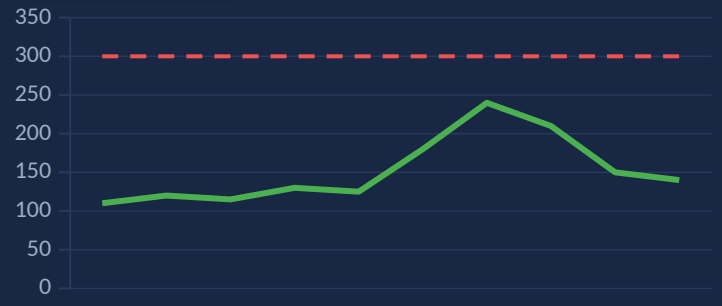
Top-feature PSI this week



Alert at 0.2

SYSTEM

p99 latency (ms)



SLO 300 ms

SYSTEM

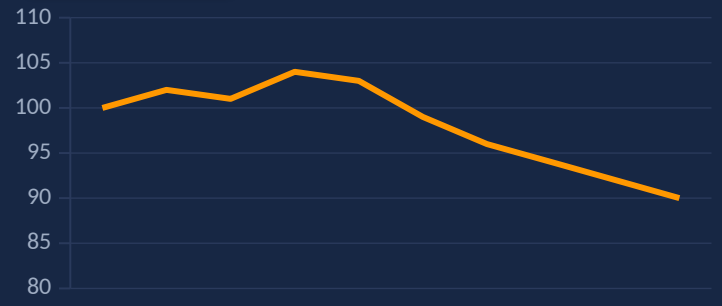
Requests & error rate



Errors < 1%

BUSINESS

Approved loans / expected loss



Watch the trend

Golden Signals, SLOs and Alert Fatigue

Measure what matters; alert only on broken promises

Latency

How fast? Track p99, not the average.

Traffic

How many requests per second?

Errors

What share of requests fail?

Saturation

How close to full is CPU or memory?

GOOGLE SRE GOLDEN SIGNALS (ABOVE) · EXAMPLE SLOs FOR THE LOAN MODEL (BELOW)

Service level objective	Pillar
AUC ≥ 0.75 on the monthly labelled sample	Model
PSI < 0.2 on the top-10 features	Data
p99 latency < 300 ms; error rate < 1%	System
Expected loss within 10% of plan	Business

Alert fatigue

If every 0.1% dip sends an email, people stop reading them. Alert on SLO breaches only; everything else stays on the dashboard.

Moving the Same Workflow to Azure ML

Trainer demonstration: no Azure account needed today

What you built today	Azure Machine Learning equivalent
main.py + config.yaml	Command job on a compute cluster
Optuna study	Sweep job (Bayesian, early termination)
Local MLflow tracking	Built-in MLflow tracking in the workspace
model.joblib in artifacts/	Model registry (versioned, tagged)
FastAPI + Docker	Managed online endpoint
PSI / KS notebook checks	Model monitoring (data drift signals)

```
job = command(code="./scripts", command="python train.py",
    environment="azureml:credit-model-env:1", compute="cpu-cluster")
```

DEMO SCRIPT · 15 MIN

1. **Workspace tour**
   Studio, compute, data assets
2. **Submit a command job**
   Same training script, cloud compute
3. **Watch metrics live**
   MLflow run in the Jobs view
4. **Register the model**
   Version, tags, lineage

Your Back-to-Work Action Plan

Five minutes, individually: commit to specific next steps

1. **One pipeline I will audit for leakage**
2. **One script I will refactor into config + modules**
3. **One prediction I will explain to a stakeholder**
4. **One drift or alert check I will add**
5. **My first step on Monday, and my deadline**

A 30-60-90 day path

30 days
Leakage audit + Pipeline on one live model

60 days
Scripts, Optuna and MLflow for that model

90 days
API, drift checks and a dashboard in place

Key Takeaways

One sentence per module to remember on Monday

MODULE 1

Trustworthy data

✓ Pick the tool from the data shape

✓ Split first; preprocess inside a Pipeline

✓ Validate the way the model will be used

MODULE 2

Training, tuning, tracking

✓ Config + seeds = reproducible runs

✓ Tune on validation folds with Optuna

✓ If it is not in MLflow, it did not happen

MODULE 3

Explain and monitor

✓ SHAP for audits, plain words for people

✓ PSI ≥ 0.2 means investigate

✓ Alert only when a human must act

MODULE 4

Serve and scale

✓ Validate input; load the model once

✓ Dashboards track 4 pillars against SLOs

✓ The same workflow runs on Azure ML

Questions? Please complete the programme feedback form before you leave.

END OF DAY 1

Thank You

From a leaky notebook to a monitored, containerised model in four hours.

Next: Day 2 · Azure ML workspace, cloud training and the model registry